COS30019 Introduction to Artificial Intelligence

Assignment 2A: Tree-Based Search

Route Finding with six search methods

Team ID: [Team ID] Tutorial group: [Group] Tutor: [Tutor name]
Submitted: 25 October 2026

Group members, contributions and signatures

Full name	Student ID	Contribution	Signature
[Member 1]	[ID]	[e.g. parser, DFS, BFS, test cases 1–5, report §1–2]	
[Member 2]	[ID]	[e.g. GBFS, A*, heuristic research, report §5]	
[Member 3]	[ID]	[e.g. CUS1, CUS2, unit tests, report §4]	
[Member 4]	[ID]	[e.g. visualiser, benchmark, visit-all research, report §6]	

Statement of contribution: each member confirms that the contribution listed above is accurate and that this report is the group's own work except where acknowledged in Section 8.

Table of contents

1 Instructions 1
1.1 Requirements 1
1.2 Running a search from the command line 1
1.3 Batch testing, unit tests and benchmark 1
1.4 Graphical visualiser 1
1.5 Notes on the implementation for the marker 2
2 Introduction 3
2.1 The Route Finding problem 3
2.2 Graph and tree concepts 3
2.3 Algorithms implemented 3
3 Features, bugs and missing items 4
3.1 Implemented features 4
3.2 Known bugs and limitations 4
4 Testing 5
5 Insights 6
5.1 Cost and moves are different goals 6
5.2 Search effort: nodes created 6
5.3 When the heuristic helps, and when it lies 7
5.4 Repeated work and scalability 7
6 Research 8
6.1 An admissible heuristic for any input 8
6.2 Scalability experiment on random graphs 8
6.3 Visiting ALL destinations with the shortest total cost 8
6.4 Tools built to study the algorithms 9
7 Conclusion 9
8 Acknowledgements and resources 9
9 References 10

List of figures

Figure 1: The visualiser after A* (AS) on test 13: 299 nodes created, cheapest path cost 17 highlighted in red 2
Figure 2: The example problem from the brief (test 1). Origin 2, destinations 4 and 5 3
Figure 3: Share of 72 random-graph runs in which each method found the lowest-cost route (left) and the fewest-moves route (right) 6
Figure 4: Nodes created by each method on five tests (log scale). Exact values are in Table 3 7
Figure 5: Median (left) and worst-case (right) nodes created on 12 random graphs per size (log scale) 7
Figure 6: Visit-all solvers: median nodes created against the number of destinations 9

List of tables

Table 1: Summary of the six implemented methods 4
Table 2: The 15 test cases, what each one tests, and the outcome 5
Table 3: Measured results: path cost / nodes created (Windows 11, Python 3.13.14). “–” means correctly reported unreachable 5
Table 4: Heuristic ablation on the tests where the scale s is below 1 (all other tests are identical) 8
Table 5: Visit-all experiment: median nodes created, and greedy baseline compared with the optimum (10 random graphs per k) 8

1 Instructions

1.1 Requirements

The program is written in **Python 3** (tested with Python 3.13.14 on Windows 11) and uses only the standard library: no search library, graph library or third-party package is needed to run `search.py`. The optional visualiser uses `tkinter`, which ships with the standard Windows installer of Python. The chart and research scripts used for this report additionally need `matplotlib`.

1.2 Running a search from the command line

Open `cmd`, change to the project folder and run either of the two equivalent commands below. `search.bat` simply forwards its two arguments to `search.py`, so the marker can use the exact `search <filename> <method>` form from the brief.

```
C:\Assignments> python search.py <filename> <method>
C:\Assignments> search <filename> <method>

C:\Assignments> python search.py tests\cases\test01_spec_example.txt AS
tests\cases\test01_spec_example.txt AS
5 9
2 3 5
```

The method name is case-insensitive and must be one of `DFS`, `BFS`, `GBFS`, `AS`, `CUS1` or `CUS2`. Output follows the required three-line format: `filename method`, then `goal number_of_nodes`, then the path from the origin to the goal. If no destination can be reached the second line reads `No goal is reachable; <number_of_nodes>`. An unknown method, a missing file or a malformed problem file produces a clear error message on standard error and a non-zero exit code instead of a Python traceback.

1.3 Batch testing, unit tests and benchmark

```
python run_tests.py [--csv results.csv]     # all 6 methods on all 15 test files
python -m unittest discover tests          # 41 automated unit tests
python benchmark_report.py                 # validates 90 runs, writes results/
python research\experiments.py             # research experiments (Section 6)
python research\visit_all.py <file> [STATE|LEGS]   # visit-all extension
```

1.4 Graphical visualiser

`python gui.py [filename] [method]` opens a window that draws the graph at its real coordinates, with one-way and two-way edges and their costs. Choose a problem file and a method, then press **Run**; **Play**, **Step** and **Back** replay the search one expansion at a time, colouring generated, expanded and current nodes, and **Show result** jumps to the final path (Figure 1). The visualiser does not change any algorithm: it wraps `problem.successors()` so that every call, which is exactly one node expansion, is recorded in order.

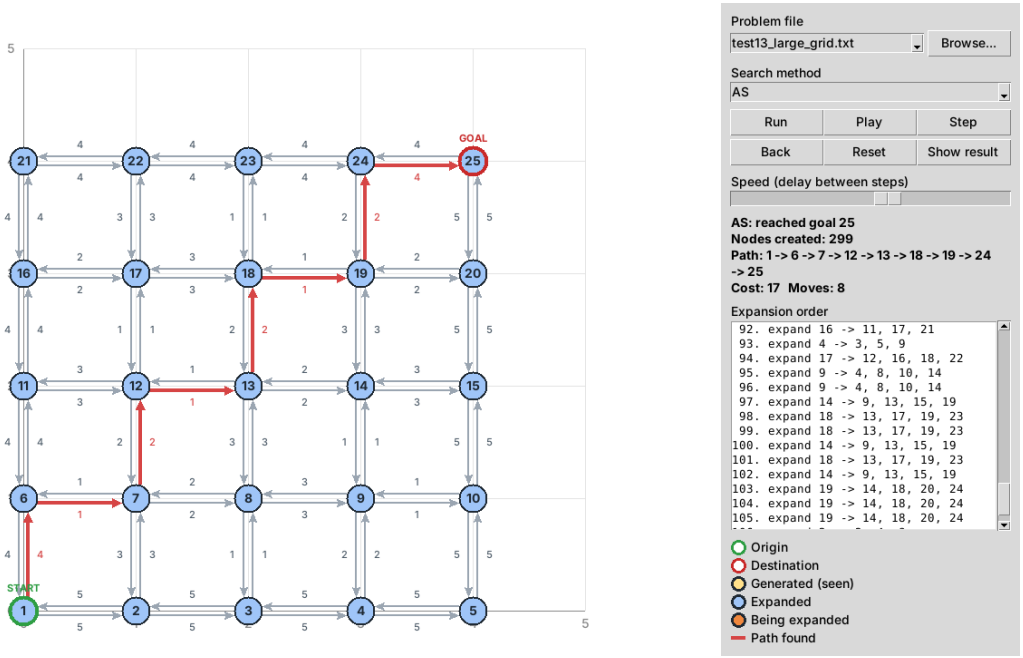


Figure 1: *The visualiser after A* (AS) on test 13: 299 nodes created, cheapest path cost 17 highlighted in red.*

1.5 Notes on the implementation for the marker

- **number_of_nodes** counts every search-tree `Node` object created, including the root and children that are later discarded as cycles or duplicates. For CUS1 and CUS2 the count includes the nodes regenerated in every iteration.
- **Tie-breaking (NOTE 2).** Successors are always generated in ascending node order. Priority queues store `(priority, insertion counter, node)`, so equal priorities are expanded in ascending node order within a family and in chronological order across branches. DFS pushes children in reverse so the smallest id is popped first.
- **Cycles.** DFS, GBFS, AS, CUS1 and CUS2 are tree searches that reject a child already on its own ancestor path (`node.on_path`). BFS keeps a global `reached` set, so it never generates the same state twice in its frontier.
- **Goal test timing.** BFS tests on generation (safe for fewest moves). DFS, GBFS and AS test when a node is removed from the frontier, which A* needs for optimality.

2 Introduction

2.1 The Route Finding problem

An agent must travel on a 2D directed, weighted graph from an **origin** node to **any one** of a set of **destination** nodes. Each node has (x, y) coordinates and each edge has a cost; an edge may be one-way, so cost(a, b) and cost(b, a) can differ or one can be missing. Figure 2 shows the example from the brief, which is also our test 1. A solution is a sequence of nodes; it may be judged by its total cost or by its number of moves (edges), and these two goals often disagree.

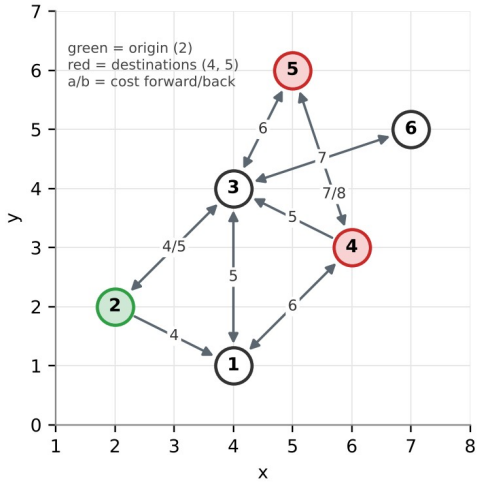


Figure 2: *The example problem from the brief (test 1). Origin 2, destinations 4 and 5.*

2.2 Graph and tree concepts

The **state space** is the graph: one state per node. A **search tree** is built on top of it: each tree node stores a state, its parent, the path cost g(n) and its depth. Because the graph has cycles (for example 1 ↔ 3), the same state can appear many times in the tree on different branches, so the tree can be much larger than the graph and is infinite if cycles are not checked. The **frontier** is the set of tree nodes generated but not yet expanded; the order in which it releases nodes is the only real difference between the six methods. **Expanding** a node generates its children from the outgoing edges. Uninformed methods use only the graph structure; informed methods also use a **heuristic** h(n), an estimate of the remaining distance to a goal. Our heuristic is the straight-line (Euclidean) distance to the nearest destination.

2.3 Algorithms implemented

Depth-first search (DFS) uses a LIFO stack and always continues the deepest branch, backing up only at dead ends. It needs memory only for the current path and its siblings, but returns the first route it meets, which can be long and expensive.

Breadth-first search (BFS) uses a FIFO queue and expands the tree level by level, so the first goal generated has the **fewest moves**. It ignores edge costs.

Greedy best-first search (GBFS) orders the frontier by h(n) only, rushing towards whatever looks closest to a goal. It is often fast but neither cost-optimal nor move-optimal.

A∗ (AS) orders the frontier by f(n) = g(n) + h(n). If h never overestimates the remaining cost (is *admissible*) and goals are tested on removal, the first goal removed is a **cheapest** route (Hart, Nilsson & Raphael 1968). Plain Euclidean distance is admissible only if every edge costs at least its length, which the file format does not guarantee. We therefore multiply h by a scale s = min(1, cost/length) over all edges, which makes h a guaranteed lower bound on any input (Section 6.1).

CUS1 – Iterative deepening DFS (uninformed). IDDFS runs a depth-limited DFS with limits 0, 1, 2, … until a goal is found (Korf 1985). It returns a fewest-moves route like BFS but stores only one path at a time, at the price of regenerating the shallow levels in every iteration. It stops early when an iteration hits no cut-off, and never needs a limit above V − 1, so unreachable goals terminate.

CUS2 – Iterative deepening A∗ for fewest moves (informed). The brief asks CUS2 to find a path with the least moves. Our IDA∗ uses f(n) = depth(n) + h_moves(n), where h_moves is the distance to the nearest destination divided by the longest edge in the graph: since one move can cover at most that distance, h_moves never overestimates the number of moves left. Each iteration is a depth-first search that prunes nodes whose f exceeds a threshold; the next threshold is the smallest f that was pruned (Korf 1985). Table 1 summarises all six methods.

Table 1: *Summary of the six implemented methods.*

Method	Frontier / evaluation	Goal test	Repeated states	Optimises	Guarantee
DFS	Stack (LIFO)	On removal	Ancestor check	Nothing	Complete on finite graphs
BFS	Queue (FIFO)	On generation	Global reached set	Moves	Fewest moves
GBFS	Priority h(n)	On removal	Ancestor check	Nothing	Complete on finite graphs
AS	Priority g(n)+s·h(n)	On removal	Ancestor check	Cost	Lowest cost
CUS1	Depth-limited DFS, limits 0,1,…	On generation in pass	Ancestor check	Moves	Fewest moves
CUS2	DFS under f = depth + h_moves threshold	When f fits threshold	Ancestor check	Moves	Fewest moves

3 Features, bugs and missing items

3.1 Implemented features

- All six required methods (DFS, BFS, GBFS, AS, CUS1, CUS2), written from scratch on a shared `Node`, `RouteProblem` and `PriorityFrontier`, with the exact output format and the NOTE 2 tie-breaking rules.
- A robust parser: accepts integer or decimal coordinates and costs, headers with content on the same line (`Origin: 2`), blank lines and any number of destinations; rejects malformed lines and undeclared nodes with a line number.
- Correct handling of special cases: origin already a destination (1 node), unreachable destinations (terminates and reports no route), one-way edges, isolated nodes.
- `search.bat` for the `search <filename> <method>` form; `run_tests.py` batch runner with CSV export; 41 unit tests; `benchmark_report.py`, which checks every result against an independent reference.
- A tkinter visualiser with step-by-step replay (Figure 1).
- Research extensions: scaled admissible heuristic, random-graph scalability study and a visit-all-destinations solver (Section 6).

3.2 Known bugs and limitations

No required feature is missing. All 90 method/test combinations and all 41 unit tests pass. The following are known limitations of the chosen design rather than wrong answers:

- DFS, GBFS, AS, CUS1 and CUS2 only check for cycles along the current path. On large, highly connected graphs the number of simple paths grows exponentially, so these methods can create millions of nodes (Section 5.4). BFS does not have this problem because of its reached set.
- CUS2's heuristic divides by the longest edge, so one long edge makes the estimate weak and IDA∗ then performs many iterations. On random graphs CUS2 created more nodes than CUS1 (Section 5.4).
- The visualiser fixes the canvas to the graph's bounding box; very large graphs (hundreds of nodes) become cluttered.

4 Testing

Testing was done at three levels. (1) **Fifteen problem files** in `tests/cases`, each built to expose one behaviour (Table 2), with the expected cheapest and fewest-moves answers worked out by hand. (2) **41 unit tests** (`python -m unittest`) covering the parser, each method's exact path and node count on chosen cases, tie-breaking, cycle handling, unreachable goals, and an IDA∗ path deeper than Python's recursion limit. (3) **An independent validator** (`benchmark_report.py`) that runs every method on every file and checks that the route starts at the origin, ends at a destination, uses only real directed edges, has no repeated node and that its reported cost matches the edges. It then compares AS against the minimum cost, and BFS, CUS1 and CUS2 against the minimum number of moves, both computed with Bellman-Ford relaxation, which shares no code with the search methods. **All 90 combinations passed.**

Table 2: *The 15 test cases, what each one tests, and the outcome.*

#	Scenario	What it checks	Result
1	Example from the brief	Two goals; spec output check	Pass
2	Origin is a goal	Stops at once, 1 node	Pass
3	Unreachable	Must terminate with no route	Pass
4	One-way edge	1→2 missing, must go 1 3 2	Pass
5	Tie-break ascending	Smaller id expanded first	Pass
6	Cheap-long vs short-expensive	Moves ≠ cost: BFS 40, AS 3	Pass
7	Multiple destinations	Nearest by cost ≠ by moves	Pass
8	Cycle trap	1→2→3→4→1 loop	Pass
9	DFS deep branch	8-edge detour vs direct edge	Pass
10	Greedy trap	GBFS cost 34, optimum 18	Pass
11	Inadmissible geometry	Edges cheaper than length	Pass
12	Dead ends	Backtracking + isolated node	Pass
13	5×5 grid, 80 edges	Effort comparison	Pass
14	Decimal values	Float coordinates and costs	Pass
15	Equal-cost tie	Two equally good goals	Pass

Table 3 lists the cost of the route returned and the number of nodes created for every method on every test. The figures quoted in Section 5 come from this table.

Table 3: *Measured results: path cost / nodes created (Windows 11, Python 3.13.14). "–" means correctly reported unreachable.*

Test	DFS	BFS	GBFS	AS	CUS1	CUS2
1	15 / 9	10 / 5	10 / 7	10 / 9	10 / 8	10 / 23
2	0 / 1	0 / 1	0 / 1	0 / 1	0 / 1	0 / 1
3	– / 3	– / 3	– / 3	– / 3	– / 3	– / 5
4	11 / 3	11 / 3	11 / 3	11 / 3	11 / 6	11 / 8
5	2 / 4	2 / 4	2 / 4	2 / 5	2 / 7	2 / 6
6	3 / 5	40 / 5	40 / 4	3 / 5	40 / 9	40 / 17
7	5 / 4	5 / 2	5 / 4	4 / 5	5 / 3	5 / 2
8	7 / 7	7 / 6	7 / 6	7 / 6	7 / 13	7 / 12
9	9 / 10	3 / 3	3 / 3	3 / 3	3 / 4	3 / 3
10	34 / 4	34 / 4	34 / 4	18 / 6	34 / 7	34 / 10
11	2 / 4	12 / 3	12 / 3	2 / 4	12 / 4	12 / 6
12	6 / 9	6 / 9	6 / 6	6 / 7	6 / 22	6 / 29
13	78 / 79	40 / 76	25 / 29	17 / 299	40 / 1421	40 / 833
14	4.4 / 4	4.4 / 4	4.4 / 4	4.4 / 4	4.4 / 7	4.4 / 15

Test	DFS	BFS	GBFS	AS	CUS1	CUS2
15	4 / 4	4 / 4	4 / 4	4 / 5	4 / 7	4 / 3

Every route found by AS has the minimum cost and every route found by BFS, CUS1 and CUS2 has the minimum number of moves. On test 1 AS returns 2 3 5 and BFS returns 2 1 4; both cost 10, so the brief's example has two optimal answers. On test 15 all methods return goal 3 (1 4 3, cost 4), which ties with 1 5 2; NOTE 2 decides it because node 4 is generated before node 5.

5 Insights

5.1 Cost and moves are different goals

The clearest lesson from the data is that “shortest” must be defined before an algorithm is chosen. In test 6 the two-move route 1 5 4 costs 40 while the three-move route 1 2 3 4 costs 3: BFS, CUS1 and CUS2 all return the 40-cost route because they are optimal for moves, and only AS returns cost 3. In test 10 GBFS is drawn towards node 2, which is close to the goal, and pays a 30-cost edge (total 34) while AS finds 18. Over 72 random graphs (Section 6.2, Figure 3) AS returned the cheapest route **100%** of the time, BFS/CUS1/CUS2 only **42%**, GBFS **40%** and DFS **7%**; conversely BFS, CUS1 and CUS2 always had the fewest moves, AS only 71%.

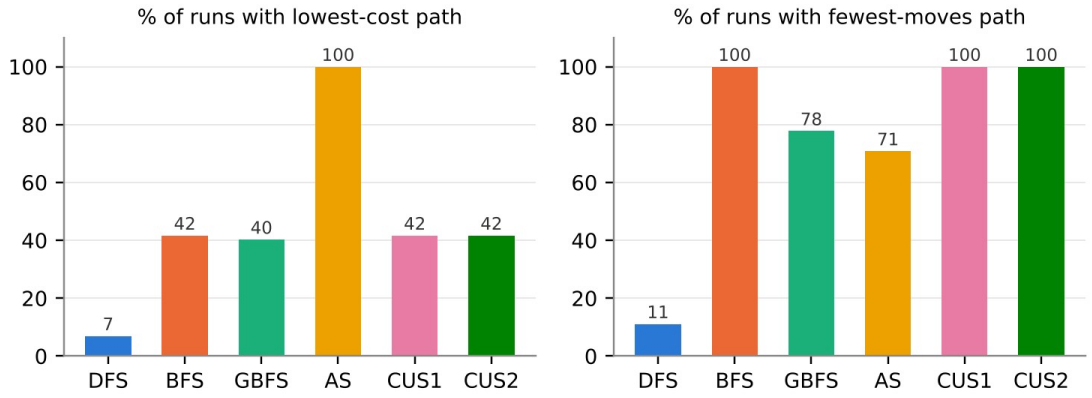


Figure 3: *Share of 72 random-graph runs in which each method found the lowest-cost route (left) and the fewest-moves route (right).*

5.2 Search effort: nodes created

Figure 4 compares the nodes created on five representative tests. On the small tests every method is cheap and the differences are a handful of nodes. GBFS is consistently the most economical: on the 5×5 grid it creates only 29 nodes, against 76 for BFS and 299 for AS, because it heads straight for the goal without reconsidering cheaper alternatives. A∗ pays for its guarantee: on the grid it has to rule out every partial route whose f is below the optimum 17, and since the tree search re-expands the same grid node via different paths, it creates four times more nodes than BFS. Wall-clock times follow node counts; every run took under 2 ms (grid: BFS 0.03 ms, AS 0.52 ms, CUS1 1.79 ms).

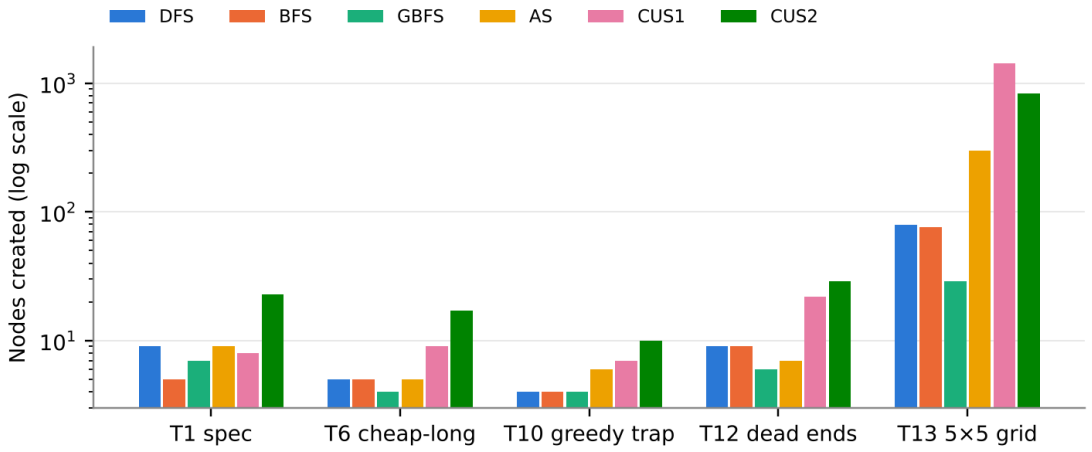


Figure 4: *Nodes created by each method on five tests (log scale). Exact values are in Table 3.*

5.3 When the heuristic helps, and when it lies

A heuristic is only safe for A∗ when it never overestimates. Test 11 places node 2 far to the side but connects it with edges of cost 1: the straight-line distance from 2 to the goal is 12.8 while the real cost is 1. With the raw heuristic, A∗ therefore believes the detour is expensive and returns the direct edge of cost 12 instead of the optimum 2; test 7 is fooled the same way (cost 5 instead of 4). Scaling h by the smallest cost/length ratio (0.078 in test 11) fixed both cases at the price of one extra node, and on the 11 tests where costs already exceed lengths the scale is 1 and nothing changes (Table 4).

5.4 Repeated work and scalability

Figure 5 shows the growth of nodes created on random graphs from 10 to 60 nodes. BFS grows almost linearly (worst case 186 nodes at n = 60) because its reached set means each node enters the frontier once. All tree-search methods grow much faster and have heavy worst cases: DFS created 352,584 nodes on one 60-node graph, AS 75,254, CUS1 888,158 and CUS2 15.3 million. The cause is the same in every case: checking cycles only along the current path still allows every simple path through the graph, and their number explodes with size. The iterative-deepening methods multiply this by regenerating the shallow levels in every iteration.

Interestingly, the informed CUS2 created **more** nodes than the uninformed CUS1 on random graphs (median 18,052 vs 2,484 at n = 60) but fewer on the regular grid (833 vs 1,421). On the grid all edges have the same length, so h_moves is accurate and prunes well. On random graphs one long edge shrinks h_moves for the whole graph, and because thresholds advance by small fractional steps, IDA∗ performs many more iterations than IDDFS's integer depth limits. A heuristic is only as good as the bound behind it.

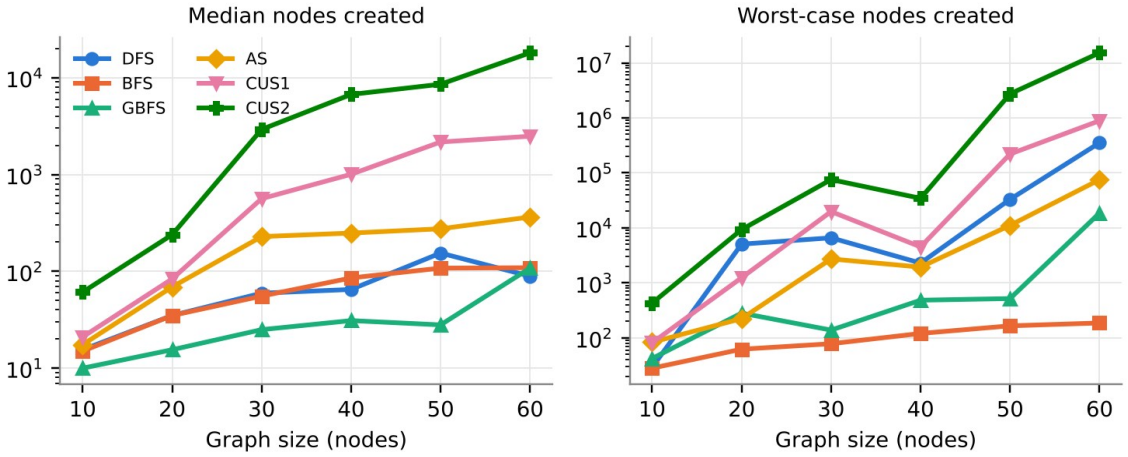


Figure 5: *Median (left) and worst-case (right) nodes created on 12 random graphs per size (log scale).*

6 Research

6.1 An admissible heuristic for any input

The brief does not require edge costs to be at least their Euclidean length, so plain straight-line distance can break A∗'s guarantee. We researched admissibility (Russell & Norvig 2021, ch. 3) and derived a fix: let s = min(1, cost(u,v)/|uv|) over all edges. Then every edge costs at least s times its length, and by the triangle inequality any route from n to a goal costs at least s·dist(n, nearest goal). So s·h is admissible on every graph the file format allows; it falls back to uniform-cost search (h = 0) only if a zero-cost edge of positive length exists. Table 4 compares the two heuristics.

Table 4: *Heuristic ablation on the tests where the scale s is below 1 (all other tests are identical).*

Test	Scale s	Optimal cost	Raw h: cost / nodes	Scaled h: cost / nodes	Raw h optimal?
4 One-way edge	0.50	11	11 / 3	11 / 3	Yes
5 Tie-break	0.71	2	2 / 4	2 / 5	Yes
7 Multiple destinations	0.49	4	5 / 4	4 / 5	No
11 Inadmissible geometry	0.08	2	12 / 3	2 / 4	No

6.2 Scalability experiment on random graphs

Fifteen hand-made tests are too small to separate the methods, so `research/experiments.py` generates random geometric graphs with fixed seeds: n nodes on a 100×100 grid, each linked to its 3 nearest neighbours, each direction kept with probability 0.8 (so some edges are one-way), cost = ⌈length × U(1, 1.6)⌉. The origin and a far, reachable destination are chosen at random. Twelve graphs were generated for each n in {10, 20, 30, 40, 50, 60} (72 graphs, 432 runs), and every run's cost and moves were compared with Bellman-Ford optimum values. The findings are reported in Figures 3 and 5 and Sections 5.1 and 5.4. Timings in this experiment were measured on a Linux machine and are used only for relative comparison.

6.3 Visiting ALL destinations with the shortest total cost

Following the research idea in the brief, `research/visit_all.py` finds the cheapest route that starts at the origin and visits **every** destination, in any order. This is a variant of the travelling salesman problem, so it cannot be solved by running A∗ to each goal in turn. We implemented two exact solvers so that each checks the other:

- **State-space A∗.** The state becomes (node, set of destinations still to visit), and the goal is any state whose set is empty. The heuristic is s × the distance to the **farthest** unvisited destination: every remaining destination must still be reached, so this never overestimates.
- **Legs + Held-Karp.** Our AS computes the cheapest leg between every ordered pair of key points (origin and destinations), then Held-Karp dynamic programming (Held & Karp 1962) chooses the cheapest order in $O(2^k \cdot k^2)$.

The main challenges were: (1) **the state space grows as V·2^k**, so pure ancestor-checking tree search repeated far too much and we had to add a best-g table (graph search) to the state-space solver; (2) **legs are directed**, so cost(a→b) ≠ cost(b→a) and the order matters in both directions; (3) **a destination can lie on the way to another**, which the state-space solver handles naturally by removing every destination it passes; (4) **some destinations are dead ends** (tests 7 and 15 have two leaf goals), so no tour exists and both solvers must report this. On test 1 both solvers return 2 1 4 5 with total cost 17. On 50 random 25-node graphs both solvers always agreed on the optimal cost (Table 5). A greedy "go to the nearest unvisited goal" baseline was worse in 14 of 50 graphs (up to 30% more expensive) and failed to find any tour in 2 graphs, because it walked into a goal from which the others were unreachable.

Table 5: *Visit-all experiment: median nodes created, and greedy baseline compared with the optimum (10 random graphs per k).*

Destinations k	State-space A* nodes	Legs + Held-Karp nodes	Greedy worse / failed	Greedy max extra cost
2	53	249	0 / 0	0%
3	108.5	844	2 / 0	8.6%
4	259.5	1094	5 / 1	23.3%

Destinations k	State-space A* nodes	Legs + Held-Karp nodes	Greedy worse / failed	Greedy max extra cost
5	275	3099	3 / 1	30.3%
6	278	3558.5	4 / 0	18.7%

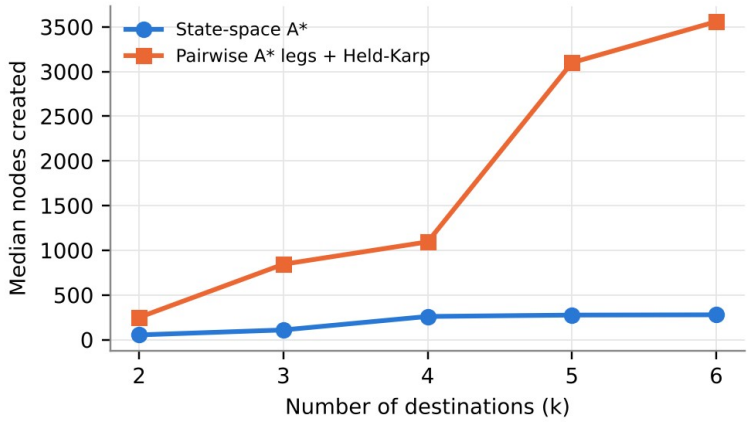


Figure 6: *Visit-all solvers: median nodes created against the number of destinations.*

State-space A∗ created 5–13 times fewer nodes because one search shares work between all orders, whereas the legs approach runs k^2 separate searches. The legs approach is simpler and reuses our tested AS unchanged, but its cost grows with k^2, and Held-Karp's 2^k table limits both methods to modest k.

6.4 Tools built to study the algorithms

Two further initiatives supported this report: the independent **Bellman-Ford validator** (Section 4), which caught a counting error during development (BFS on test 1 was first reported as 4 nodes; a generated duplicate also creates a node, so the correct count is 5), and the **step-by-step visualiser** (Section 1.4), which made the difference between expansion orders visible and helped us debug tie-breaking.

7 Conclusion

For the Route Finding problem as specified, where the goal is the lowest-cost route, **A∗ with an admissible heuristic is the best choice**: it was the only method that returned the cheapest route on every test and every random graph, and with our scaled heuristic this guarantee holds even when edge costs are smaller than their geometric length. If the number of moves is what matters, **BFS** is the best choice: it is move-optimal and, thanks to its reached set, by far the most predictable in effort. IDDFS and IDA∗ give the same answers with less memory but much more repeated work at this scale. GBFS is the fastest when any reasonable route will do, and DFS should be avoided for this problem because its routes were the most expensive (on average 1.6–2.4× optimal).

The data also shows how performance could be improved. The largest gain would come from replacing ancestor-only cycle checking in A∗ with a closed set or best-g table, as we did for the visit-all solver: this keeps optimality with a consistent heuristic and would remove the exponential worst cases in Figure 5. A tighter heuristic, for example per-region scale factors or precomputed landmark distances, would reduce A∗'s node count further, and bidirectional search would help on large graphs with a single destination.

8 Acknowledgements and resources

- **COS30019 lecture and tutorial material (Swinburne).** Definitions of DFS, BFS, GBFS and A∗, the frontier/explored-set terminology and the tie-breaking conventions.
- **Russell & Norvig, Artificial Intelligence: A Modern Approach, and the aima-python repository** (github.com/aimacode/aima-python). Used to understand node/problem abstractions and admissibility; our code was written from scratch and does not import it.
- **Korf (1985).** The design of IDDFS (CUS1) and IDA∗ (CUS2), including advancing the threshold to the smallest pruned f.
- **Python documentation** (docs.python.org): `heapq` for the priority frontier, `itertools.count` for the insertion counter, `tkinter` for the visualiser and `unittest` for testing.

- **Matplotlib documentation** (matplotlib.org): used to produce the charts in this report.
- **Generative AI acknowledgement.** Claude (Anthropic) was used to help draft this report from our code, test files and benchmark output, to generate the charts, and to write the research scripts in `research/` (random-graph experiments and the visit-all solvers), which the team reviewed and ran. [Team to complete: describe any other AI use during coding, or state that none was used.]

9 References

Hart, P.E., Nilsson, N.J. & Raphael, B. 1968, 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107.

Held, M. & Karp, R.M. 1962, 'A dynamic programming approach to sequencing problems', Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, pp. 196–210.

Korf, R.E. 1985, 'Depth-first iterative-deepening: an optimal admissible tree search', Artificial Intelligence, vol. 27, no. 1, pp. 97–109.

Python Software Foundation 2025, heapq — Heap queue algorithm, Python 3 documentation, viewed October 2026, <https://docs.python.org/3/library/heapq.html>.

Python Software Foundation 2025, tkinter — Python interface to Tcl/Tk, Python 3 documentation, viewed October 2026, <https://docs.python.org/3/library/tkinter.html>.

Russell, S. & Norvig, P. 2021, Artificial Intelligence: A Modern Approach, 4th edn, Pearson, Hoboken, NJ.

Swinburne University of Technology 2026, COS30019 Introduction to AI, Assignment 2 Part A: Tree Based Search, unit material.